

## Лабораторна робота №6

### Розробка програм з користувацькими класами. Робота з класами та об'єктами.

Виконав: студент групи ALK-43

Вівчар Вадим Вікторович

#### Мета роботи

Навчитися створювати власні класи у мові C++, працювати з конструкторами, деструктором, методами та перевантаженням операторів. Закріпити практичні навички об'єктно-орієнтованого програмування.

#### Завдання 1. Клас Fraction (робота з дробами)

Мета: створити клас для роботи з раціональними дробами. Реалізувати перевантаження операторів +, -, \*, /, а також конструктори та деструктор.

#### Код програми

```
#include <iostream>
#include <stdexcept>
#include <cstdlib>

long long my_gcd(long long a, long long b) {
    a = llabs(a); b = llabs(b);
    while (b != 0) { long long t = b; b = a % b; a = t; }
    return (a == 0 ? 1 : a);
}

class Fraction {
private:
    long long num, den;
    void normalize() {
        if (den == 0) throw std::invalid_argument("Denominator cannot be zero");
        if (den < 0) { num = -num; den = -den; }
        long long g = my_gcd(num, den);
        num /= g; den /= g;
    }
public:
```

```

Fraction(): num(0), den(1) {}
Fraction(long long n, long long d): num(n), den(d) { normalize(); }
~Fraction() {}

void read() {
    std::cout << "Enter numerator and denominator (separated by space): ";
    std::cin >> num >> den;
    normalize();
}

void print() const {
    std::cout << num << "/" << den;
}

double value() const { return static_cast<double>(num) / den; }

Fraction operator+(const Fraction& other) const {
    return Fraction(num*other.den + other.num*den, den*other.den);
}
Fraction operator-(const Fraction& other) const {
    return Fraction(num*other.den - other.num*den, den*other.den);
}
Fraction operator*(const Fraction& other) const {
    return Fraction(num*other.num, den*other.den);
}
Fraction operator/(const Fraction& other) const {
    return Fraction(num*other.den, den*other.num);
}
};

int main() {
    Fraction a, b;
    a.read();
    b = Fraction(3,4);
    std::cout << "a = "; a.print(); std::cout << " ~ " << a.value() << "\n";
    std::cout << "b = "; b.print(); std::cout << " ~ " << b.value() << "\n\n";
    Fraction res;
    res = a + b; std::cout << "a + b = "; res.print(); std::cout << " ~ " << res.value() << "\n";
    res = a - b; std::cout << "a - b = "; res.print(); std::cout << " ~ " << res.value() << "\n";
    res = a * b; std::cout << "a * b = "; res.print(); std::cout << " ~ " << res.value() << "\n";
    res = a / b; std::cout << "a / b = "; res.print(); std::cout << " ~ " << res.value() << "\n";
    return 0;
}

```

Результати виконання програми:

```

a = 5/9 ~ 0.555556
b = 3/4 ~ 0.75
a + b = 47/36 ~ 1.30556
a - b = -7/36 ~ -0.194444
a * b = 5/12 ~ 0.416667
a / b = 20/27 ~ 0.740741

```

Скріншоти результатів:

```
D:\work\Project23.exe
Enter numerator and denominator (separated by space): 5 9
a = 5/9 ~ 0.555556
b = 3/4 ~ 0.75

a + b = 47/36 ~ 1.30556
a - b = -7/36 ~ -0.194444
a * b = 5/12 ~ 0.416667
a / b = 20/27 ~ 0.740741

-----
Process exited after 4.642 seconds with return value 0
Press any key to continue . . .
```

## Завдання 2. Клас TimeOfDay (робота з часом)

Мета: створити клас для представлення часу в межах доби. Реалізувати методи введення, виведення, обчислення часу до кінця доби та перевантаження операторів + і - для зміни часу на певну кількість секунд.

### Код програми

```
#include <iostream>
#include <stdexcept>
#include <iomanip>

class TimeOfDay {
private:
    int h, m, s;
    static int toTotal(int hh, int mm, int ss) {
        if (hh < 0 || hh > 23 || mm < 0 || mm > 59 || ss < 0 || ss > 59)
            throw std::out_of_range("Invalid time");
        return hh * 3600 + mm * 60 + ss;
    }
    static void fromTotal(int total, int &hh, int &mm, int &ss) {
        if (total < 0 || total > 86399)
            throw std::out_of_range("Out of range");
        hh = total / 3600;
        mm = (total % 3600) / 60;
        ss = total % 60;
    }
public:
    TimeOfDay(): h(0), m(0), s(0) {}
    TimeOfDay(int hh, int mm, int ss) { set(hh, mm, ss); }
    ~TimeOfDay() {}

    void set(int hh, int mm, int ss) {
        (void)toTotal(hh, mm, ss);
        h = hh; m = mm; s = ss;
    }

    void read() {
```

```

        std::cout << "Enter time (hours minutes seconds): ";
        int hh, mm, ss;
        std::cin >> hh >> mm >> ss;
        set(hh, mm, ss);
    }

    void print() const {
        std::cout << std::setfill('0') << std::setw(2) << h << ":"
            << std::setw(2) << m << ":" << std::setw(2) << s;
    }

    void timeLeft(int &H, int &M, int &S) const {
        int total = toTotal(h, m, s);
        int delta = 86400 - 1 - total;
        if (delta < 0) delta = 0;
        fromTotal(delta, H, M, S);
    }

    TimeOfDay operator+(int sec) const {
        int total = toTotal(h, m, s) + sec;
        if (total < 0 || total > 86399)
            throw std::out_of_range("Resulting time out of range");
        int hh, mm, ss;
        fromTotal(total, hh, mm, ss);
        return TimeOfDay(hh, mm, ss);
    }

    TimeOfDay operator-(int sec) const { return (*this) + (-sec); }
};

int main() {
    std::cout << "=== TimeOfDay Class Demo ===\n";
    TimeOfDay t;
    t.read();

    std::cout << "Current time: ";
    t.print();
    std::cout << "\n";

    int H, M, S;
    t.timeLeft(H, M, S);
    std::cout << "Time left until midnight: "
        << std::setfill('0') << std::setw(2) << H << ":"
        << std::setw(2) << M << ":" << std::setw(2) << S << "\n";

    TimeOfDay t2 = t + 75;
    std::cout << "t + 75 sec = ";
    t2.print();
    std::cout << "\n";

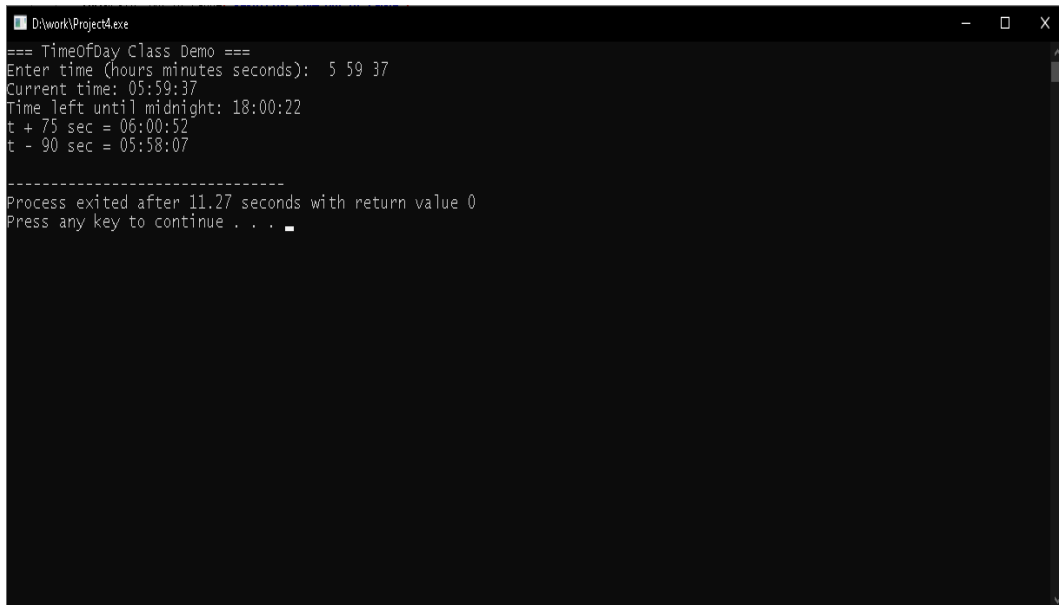
    TimeOfDay t3 = t - 90;
    std::cout << "t - 90 sec = ";
    t3.print();
    std::cout << "\n";
    return 0;
}

```

Результати виконання програми:  
 Current time: 05:59:37

Time left until midnight: 18:00:22  
t + 75 sec = 06:00:52  
t - 90 sec = 05:58:07

Скріншоти результатів:



```
=== TimeOfDay Class Demo ===
Enter time (hours minutes seconds): 5 59 37
Current time: 05:59:37
Time left until midnight: 18:00:22
t + 75 sec = 06:00:52
t - 90 sec = 05:58:07

-----
Process exited after 11.27 seconds with return value 0
Press any key to continue . . .
```

Контрольні запитання

1. Що називають полями класу? Поля класу — це змінні, що описують стан об'єкта (його характеристики, властивості).
2. Що називають методами класу? Методи — це функції-члени, які визначають поведінку об'єктів класу і оперують його полями.
3. Якими властивостями володіють класи? Основні властивості класів: інкапсуляція, наслідування та поліморфізм.
4. Що таке наслідування класів? Наслідування — це механізм, що дозволяє створювати нові класи на основі вже існуючих, успадковуючи їхні поля та методи.
5. Що називають поліморфізмом методів? Поліморфізм — це здатність методів з однаковим іменем виконувати різні дії залежно від типу або класу об'єкта.
6. Як у C++ реалізується перевантаження операторів? Перевантаження операторів у C++ здійснюється шляхом визначення спеціальних функцій із ключовим словом `operator`.
7. Для чого призначений конструктор класу? Конструктор використовується для ініціалізації об'єктів при їх створенні — встановлює початкові значення полів.